

DIGITAL ASSIGNMENT: 2

Vatsal Pareek

25 BAI1119

FI+TFI

DDL Commands:

```
CREATE TABLE USERS(  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(120) UNIQUE NOT NULL,  
    phone VARCHAR(15),  
    password_hash VARCHAR(256) NOT NULL,  
    user_type VARCHAR(10) NOT NULL,  
    CONSTRAINT chk_user_type CHECK (usertype in (  
        'CUSTOMER', 'SELLER'))  
); AUTO_INCREMENT = 1000;  
  
CREATE TABLE SELLERS(  
    user_id INT PRIMARY KEY,  
    gstin VARCHAR(15) UNIQUE NOT NULL,  
    company_name VARCHAR(150) NOT NULL,  
    bank_acc_no VARCHAR(30) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES USERS (user_id)  
    ON DELETE RE CASCADE  
);  
  
CREATE TABLE CUSTOMERS(  
    user_id INT PRIMARY KEY,  
    street VARCHAR(200) NOT NULL,  
    pincode VARCHAR(10) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES USERS (user_id)  
    ON DELETE CASCADE  
);
```

```
CREATE TABLE CATEGORIES(  
  category_id INT AUTO-INCREMENT PRIMARY KEY,  
  category_name VARCHAR(100) NOT NULL,  
  parent_category_id INT NULL,  
  FOREIGN KEY (parent_category_id) REFERENCES  
    CATEGORIES (category_id) ON DELETE SET NULL  
) AUTO-INCREMENT = 100;
```

```
CREATE TABLE PRODUCTS(  
  product_id INT AUTO-INCREMENT PRIMARY KEY,  
  seller_id INT NOT NULL,  
  category_id INT NOT NULL,  
  product_name VARCHAR(150) NOT NULL,  
  unit_price DECIMAL(10,2) NOT NULL,  
  stock_qty INT NOT NULL,  
  CONSTRAINT chk_unit_price CHECK (unit_price >= 0),  
  CONSTRAINT chk_stock_qty CHECK (stock_qty >= 0),  
  FOREIGN KEY (seller_id) REFERENCES SELLERS (user_id)  
    ON DELETE CASCADE,  
  FOREIGN KEY (category_id) REFERENCES CATEGORIES  
    (category_id)  
) AUTO-INCREMENT = 500;
```

```
CREATE TABLE ORDERS(  
  order_id INT AUTO-INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATETIME DEFAULT CURRENT_TIMESTAMP  
    NOT NULL,  
  order_status VARCHAR(20) DEFAULT 'PENDING',  
  total_amount DECIMAL(12,2) DEFAULT 0.00 NOT NULL,  
  CONSTRAINT chk_order_status CHECK (order_status IN  
    ('PENDING', 'PAID', 'SHIPPED', 'DELIVERED', 'CANCELLED')),  
  CONSTRAINT chk_total_amount CHECK (total_amount >= 0),  
  FOREIGN KEY (customer_id) REFERENCES CUSTOMER (user_id)  
    ON DELETE CASCADE
```

) AUTO_INCREMENT = 1;

CREATE TABLE ORDER_ITEMS(

order-id INT NOT NULL,

item-id INT NOT NULL,

product-id INT NOT NULL,

quantity INT NOT NULL,

item-price DECIMAL(10,2) NOT NULL,

PRIMARY KEY(order-id, item-id),

CONSTRAINT chk_quantity CHECK(quantity > 0),

CONSTRAINT chk_item_price CHECK(item-price >= 0),

FOREIGN KEY(order-id) REFERENCES ORDERS(order-id)

ON DELETE CASCADE,

FOREIGN KEY(product-id) REFERENCES PRODUCTS
(product-id)

);

CREATE TABLE PAYMENTS(

payment-id INT AUTO_INCREMENT PRIMARY KEY,

order-id INT UNIQUE NOT NULL,

payment-date DATETIME DEFAULT CURRENT_TIMESTAMP
NOT NULL,

amount DECIMAL(12,2) NOT NULL,

payment-method VARCHAR(20) NOT NULL,

payment-status VARCHAR(20) DEFAULT 'SUCCESS',

CONSTRAINT chk_amount CHECK(amount >= 0),

CONSTRAINT chk_payment_method CHECK(payment-method
IN ('CREDIT-CARD', 'DEBIT-CARD', 'UPI', 'NET-BANKING', 'COD'))

CONSTRAINT chk_payment_status REFERENCES ORDERS
(order-id) ON DELETE CASCADE

) AUTO_INCREMENT = 1;

CREATE TABLE SHIPMENTS(

shipment-id INT AUTO_INCREMENT PRIMARY KEY,

order-id INT UNIQUE NOT NULL

tracking-number VARCHAR(50) UNIQUE NOT NULL,

carrier VARCHAR(80) NOT NULL,

ship-date DATETIME DEFAULT CURRENTTIMESTAMP;
delivery-status VARCHAR(20) DEFAULT 'IN-TRANSIT';
CONSTRAINT chk-delivery-status CHECK (delivery-status
IN ('PENDING', 'IN-TRANSIT', 'DELIVERED', 'RETURNED'));
FOREIGN KEY (order-id) REFERENCES ORDERS (order-id)
ON DELETE CASCADE
) AUTO-INCREMENT=1;

PROCEDURAL Logic:-

```
CREATE TRIGGER trig-order-total_insert  
AFTER INSERT ON ORDER_ITEMS  
FOR EACH ROW  
BEGIN
```

```
    UPDATE ORDERS
```

```
    SET total_amount = total_amount + (NEW.quantity +  
    NEW.item_price)
```

```
    WHERE order_id = NEW.order_id;
```

```
END$$
```

```
CREATE TRIGGER trig-order-total_update  
AFTER UPDATE ON ORDER_ITEMS  
FOR EACH ROW  
BEGIN
```

```
    UPDATE ORDERS
```

```
    SET total_amount = total_amount - (OLD.quantity *  
    OLD.item_price) + (NEW.quantity * NEW.item_price)
```

```
    WHERE order_id = NEW.order_id;
```

```
END$$
```

```
CREATE TRIGGER trig-order-total_delete  
AFTER DELETE ON ORDER_ITEMS  
FOR EACH ROW  
BEGIN
```

```
    UPDATE ORDERS
```

```
    SET total_amount = total_amount - (OLD.quantity * OLD.item_price)
```

```
    WHERE order_id = OLD.order_id;
```

```
END$$
```

```
CREATE TRIGGER - trig-check-and-reduce-stock  
BEFORE INSERT ON ORDER_ITEMS  
FOR EACH ROW  
BEGIN
```

```
    DECLARE v-available_qty INT;
```

```
    SELECT stock_qty INTO v-available_qty  
    FROM PRODUCTS
```

```
WHERE product_id = NEW.product_id
FOR UPDATE;

IF v-available_qty < NEW.quantity THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Insufficient stock available for
        Product ID';
END IF;

UPDATE PRODUCTS
SET stock_qty = stock_qty - NEW.quantity
WHERE product_id = NEW.product_id;
END $$
```

```
CREATE PROCEDURE sq-place_order(
    IN p-customer_id INT,
    IN p-product_id INT,
    IN p-quantity INT,
    IN p-payment-method VARCHAR(20),
    OUT p-order_id INT
)
BEGIN
    DECLARE v-unit-price DECIMAL(10,2);
    DECLARE v-order_id INT;
    DECLARE v-tot_amt DECIMAL(12,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;
    SELECT unit-price INTO v-unit-price
    FROM PRODUCTS
    WHERE product_id = p-product_id;
    INSERT INTO ORDERS (customer_id, order_status, total_amount)
    VALUES (p-customer, 'PENDING', 0.00);
```

```

SET v-order-id = LAST_INSERT_ID();
INSERT INTO ORDER_ITEMS (order-id, item-id, product-id,
    quantity, item-price)
VALUES (v-order-id, 1, p-product-id, p-quantity, v-unit-price);
SELECT total-amount INTO v-tot-amt
FROM ORDERS
WHERE order-id = v-order-id;
INSERT INTO PAYMENTS (order-id, amount, payment-method,
    payment-status)
VALUES (v-order-id, v-tot-amt, p-payment-method, 'success');
UPDATE ORDERS
SET order-status = 'PAID'
WHERE order-id = v-order-id;
INSERT INTO SHIPMENTS (order-id, tracking-number, carrier,
    delivery-status)
VALUES (v-order-id, CONCAT('TRK-', DATE_FORMAT(NOW(),
    '%Y%m%d'), '-', v-order-id), 'BlueDart', 'PENDING');
SET p-order-id = v-order-id;
COMMIT;
END $$
DELIMITER;

```

DEMONSTRATION SCRIPT

```

INSERT INTO USERS (name, email, phone, password-hash,
    user-type)
VALUES ('Alice Sharma', 'alice@gmail.com', '9847001234',
    'hash-alice-pwd', 'CUSTOMER');
INSERT INTO CUSTOMERS (user-id, street, pincode)
VALUES (seq-user-id; CURRVAL, '14 Main', '560038');
INSERT INTO USERS (name, email, phone, password-hash,
    user-type)
VALUES ('Apex', 'contain@gmail.com', '9018346767', 'hash-apex-pwd',

```

SELLER');

```
INSERT INTO SELLERS (user-id, gstn, company-name, bank-acc.no)
VALUES (seq-user-id.CURRVAL, '29AA00A1Z5', 'Apex', '901236712
(34567));
```

```
INSERT INTO CATEGORIES (Category-Name, parent-category-id)
VALUES ('Electronics', NULL);
```

```
INSERT INTO CATEGORIES (Category-name, parent_category-id)
VALUES ('Laptops', seq-category-id.CURRVAL);
```

```
INSERT INTO PRODUCT (seller-id, category-id, product-name,
unit-price, stock-qty)
```

```
VALUES (
```

```
(SELECT user-id FROM USERS WHERE email = 'contact@gmail.com'),
```

```
(SELECT category-id FROM CATEGORIES WHERE
category-name = 'Laptops'),
```

```
'ProBook 16GB', 55000.00, 15
);
```

```
COMMIT;
```

```
VARIABLE new-order-id NUMBER;
```

```
DECLARE
```

```
v-customer-id INT;
```

```
v-product-id INT
```

```
BEGIN
```

```
SELECT user-id INTO v-customer-id FROM USERS WHERE
email = 'alice@gmail.com';
```

```
SELECT product-id INTO v-product-id FROM PRODUCTS WHERE
product-name = 'ProBook 16GB';
```

```
sp-place_order(
```

```
p-customer-id => v-customer-id,
```

```
p-product-id => v-product-id;
```

```
p-quantity => 2,
```

```
p-payment-method => 'UPI';
```

```
p-order-id => new-order-id
```

```
END;
```